NetTower – Network Situational Awareness Tool

Benjamin Molloy

ASE-485 Capstone FP

Problem Domain

“Small or disrupted networks lack a simple way to quickly understand what devices are present and how they are generally connected.”

- Limited visibility in small or disrupted networks
- No centralized monitoring in home labs / ad-hoc environments
- Enterprise tools are complex and resource-heavy
- Difficult to quickly identify:
  - Devices on the network
  - Reachability
  - High-level connectivity

Solution

“NetTower provides a lightweight, quickly deployable system that discovers reachable devices, infers high-level connectivity relationships, and presents the results through an intuitive network visualization.”

- Lightweight, agentless network discovery system
- Identifies reachable devices on a network
- Infers high-level connectivity relationships
- Presents results through an intuitive topology view
- Designed for clarity and accessibility over deep analysis

Solution Overview (Tech Stack)

Languages

- Python (Backend)
- JavaScript, HTML, CSS (Frontend)

Core Components

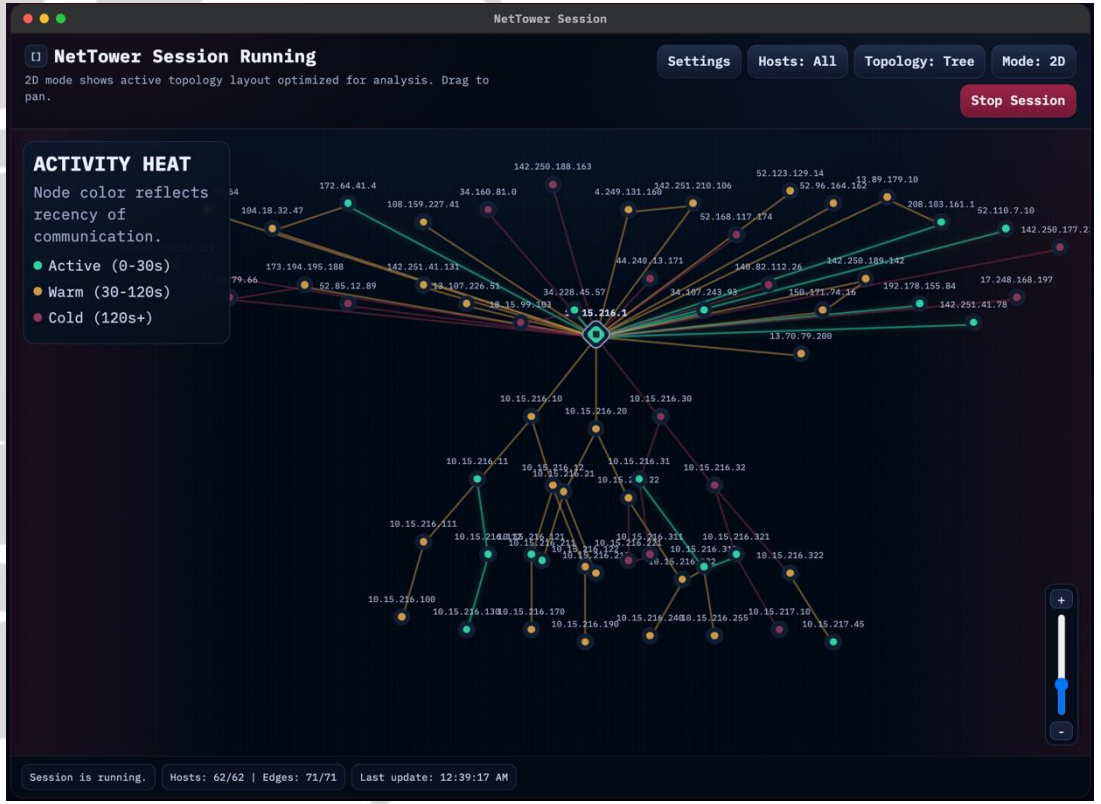
- Discovery (Active + Passive Scanning)
- Event-driven processing pipeline
- Data modeling (Hosts & Edges)
- MongoDB (runtime storage)
- Electron-based visualization interface

Networking Tools

- tcpdump
- ICMP / ARP utilities
- traceroute
- nmap

Demo (Video/Product Page)

- Demo Vid (OneDrive)
- Demo Vid (GitHub)
- https://nettower.org
- https://molloy.info



Canvas: Progress Link
GitHub: Progress Link

Sprint Progress

Sprint 1 (Backend Development)

- Project setup and baseline discovery
- Host identification and scanning
- Connectivity inference and relationship modeling
- Backend pipeline completion and stabilization

Sprint 2 (Frontend + Integration)

- Frontend development and visualization
- UI refinement and enhancements
- Backend–frontend integration
- Performance optimization and testing
- Documentation and final deployment

AI Use (Coding + Learning)

AI for Development

- Assisted with architecture refinement
- Helped debug and validate implementation decisions
- Accelerated iteration and problem-solving

Learning with AI

- Layer 2–4 protocol behavior and limitations
- Packet structure (Ethernet, IP, TCP/UDP, ICMP)
- Understanding what can vs cannot be inferred from network data
- Validating reasoning in constrained environments

Burn Down / Key Numbers

Key Numbers

- Total Lines of Code: 18,652
- Core Features: 8/8
- Requirements: 17/17

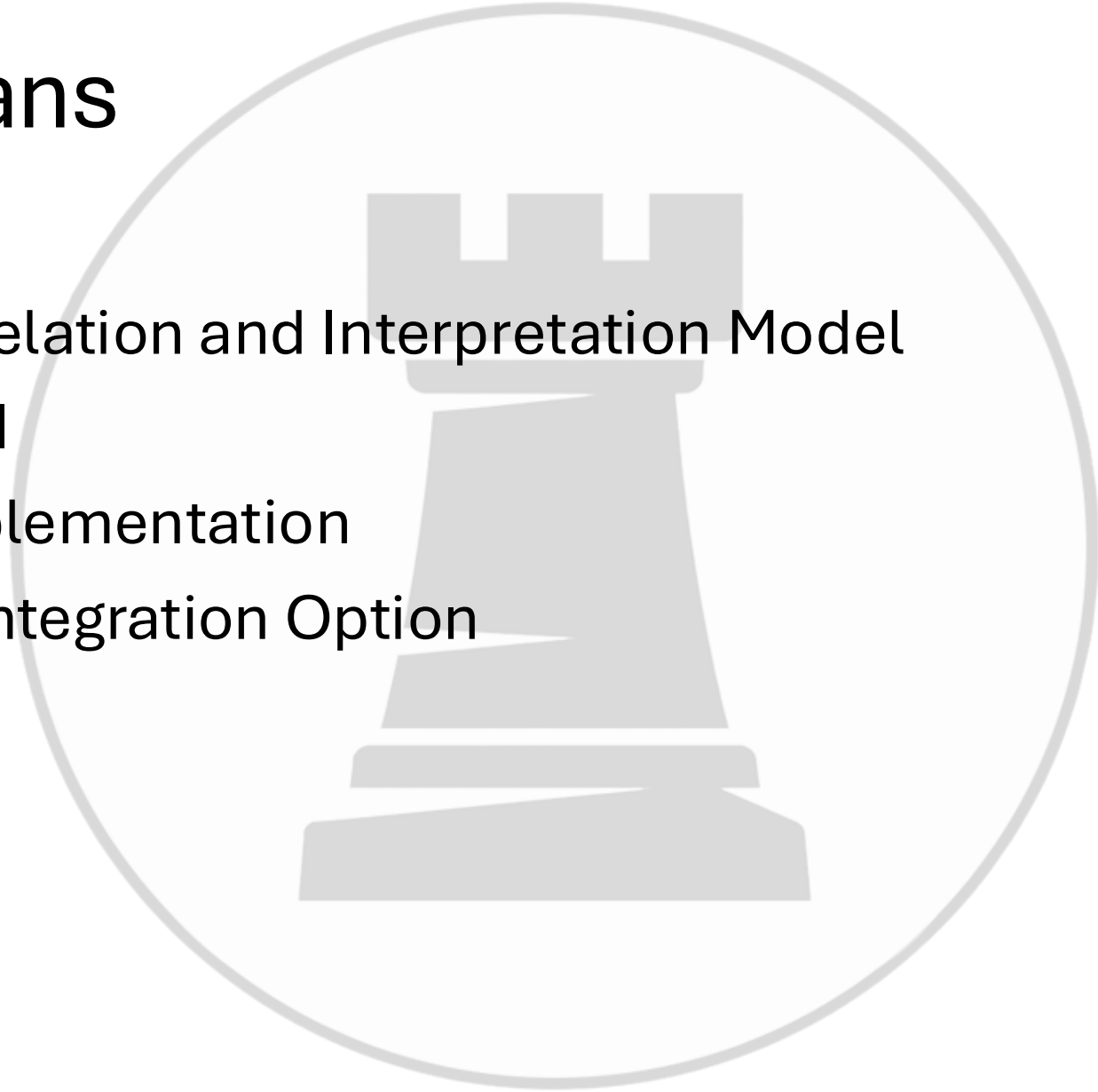
Code Tests

- Automated Tests: 28/28 Passed
- Manual Tests: 6/6 Passed
- Total Tests: 34/34 Passed

100% Burn Down Rate

Future Plans

- Rework Correlation and Interpretation Model
- Modernize UI
- Platform Implementation
- Host Agent Integration Option




Questions?